Tecnológico Nacional de México

Instituto Tecnológico de La Laguna

Inteligencia Artificial
INGENIERÍA EN SISTEMAS COMPUTACIONALES

Proyecto <<Unidad 1>>

Presenta:

Enrique Sanchez Morales	21130936
Ximena Arciniega Ochoa	23130025
Luis Gael Fernandez Davalos	23130481
Sergio Yael Perea Flores	23130031

Torreón,Coahuila
Septiembre 2026

Índice

Introducción **3**
Desarrollo **3**
Introducción y Arquitectura del Agente C.C. t0 3
Atributos y Estado Interno 5
Modelado y Generación del Entorno 7
Estructura Matricial del Mundo 7
Posicionamiento del Altar y Fragmentos 8
Renderizado Pixel-Art por Texturas 9
Ciclo de Decisiones: Percepción, Regla de Reflejo y Acción 11
Percepción Inmediata 11
Tabla Jerárquica de Condición-Acción 12
Módulo de Actuar 13
Representación Gráfica del Agente 14
Mapeo del Sprite de C.C. 14
Retroalimentación Visual de Carga 16
Bucle de Simulación e Interfaz 16
Bucle de Ejecución Asíncrona 16
Sincronización de la Interfaz de Usuario (UI) 17
Control de Inicio y detenido de la Simulación 19
Arquitectura agente t-1 22
Resumen 22
Archivos nuevos 23
agente.js 23
Los 5 sentidos del Agente 24
Vista 24
Transformación a Coordenadas Absolutas 25
Validación de Fronteras del Mapa 25
Inspección de Contenido y Detección del Dragón 25
Marcado de Peligro 25
Oído 26
Cálculo Matemático: Distancia Manhattan 26
Resolución Robusta de Coordenadas 27
Clasificación de Intensidad 27
Estructura del Objeto de Percepción 27
Olfato 27
Escaneo del Mapa y Filtrado de Cristales 28
Cálculo de Distancias Manhattan 28
Identificación del Fragmento Más Cercano 28
Fórmula Matemática de Intensidad del Olfato 28
Retorno de Datos 29
Tacto 30
Lectura del Terreno Actual 30

Evaluación de Resistencia y Fricción 30
Función Integrada de Validación de Límites 30
Estructura del Objeto de Retorno 31
Gusto 31
Validación de Presencia en el Altar 31
Coeficiente de Eficiencia 32
Cálculo de la Energía Potencial a Recargar 32
Estructura del Objeto de Retorno 33
Memoria y peligro 33
Decisión 33
Daño, muerte y respawn 34
Movimiento y dibujo 34
Entidades nuevas 34
mapa.js 35
main.js 35
index.html y styles.css 36
index.html 36
styles.css 36
Explicación Obstáculos: 36
El Dragón (obstáculo móvil) 37
Claire, la aliada (obstáculo estático benéfico) 39
El Castillo (obstáculo bloqueante grande) 39
El Brujo (obstáculo estático dañino) 39
La Trampa de hielo (obstáculo oculto) 40
Los Árboles (muros y corredores en "L") 41
El río de hielo y el hielo que se rompe 41
cambios realizados en los agentes 43
Problemas 43
Conclusiones 46
Referencias bibliográficas 46

Introducción

Este proyecto trata sobre la creación de un agente t0, sólo vive en el presente y no guarda información, a un t-1 el cual puede guardar información sobre; el terreno, costo de cada paso, objetivos, centro de carga e incluso si otro agente llego a hacerle daño.

En mundo en el cual se desarrollara nuestro agente llamada “C.C”, de una serie llamada *Code Geass, se adjuntará una imagen del personaje abajo para tener un poco de información porque en pixel art no se distingue bien, este mundo es discreto* y se desarrolla en un ambiente nevado, un bosque invernal, el cual cuenta con arboles, casillas de hielo, nieve, enemigo (el hechicero y un dragón), asi como una aliada que le ayudará a combatirlos en su camino para recolectar fragmentos del espejo el cual le mostrará su origen.

Desarrollo

Introducción y Arquitectura del Agente C.C. t0

El Agente C.C en su versión t0 se caracteriza por ser un Agente de Reflejo Simple. Esto quiere decir que solo toma decisiones en el instante del presente, ignorando el historial de percepciones pasadas y careciendo de una memoria.

Su flujo de procesamiento no almacena información de casillas visitadas anteriormente ni planifica rutas a largo plazo.

Percepción puntual: El método percibir(mapa) evalúa únicamente la celda actual donde se encuentra posicionada C.C. en la cuadrícula.

```
percibir(mapa) {
    const casillaActual = mapa.grid[this.y][this.x];
    return {
        hayFragmento: casillaActual === TIPO_CASILLA.FRAGMENTO,
        enBase: casillaActual === TIPO_CASILLA.BASE_ESPEJO,
        energiaIncompleta: this.energia < 100
    };
}
```

Imagen 1.1 Método del agente para percibir el mapa

Evaluación de reglas condición-acción: La función reglaReflejo(percepcion) procesa los datos percibidos mediante un conjunto jerárquico de declaraciones condicionales (if) para determinar la respuesta inmediata.

```
reglaReflejo(percepcion) {
    if (this.energia <= 0) {
    return 'SIN_ENERGIA';
}
    if (percepcion.hayFragmento && !this.tieneFragmento) {
        return 'RECOGER_FRAGMENTO';
    }
    if (percepcion.enBase && this.tieneFragmento) {
        return 'DEPOSITAR_FRAGMENTO';
    }
    if (percepcion.enBase && percepcion.energiaIncompleta) {
        return 'RECARGAR';
    }
    return 'MOVER_ALEATORIO';
}
```

Imagen 1.2 Método condicionales del Agente para respuesta inmediata

Ausencia de memoria de estado: El agente no sabe en qué dirección caminó en el turno anterior ni dónde se encuentran los demás fragmentos dispersos. Si la regla determina movimiento, este se realiza hacia una dirección adyacente elegida de forma completamente aleatoria.

Atributos y Estado Interno

A pesar de que el agente no posea memoria espacial, requiere de variables de estado interno dentro de su clase (Agente) para poder modelar sus limitaciones físicas y el progreso de su objetivo en la simulación.

Posición en la cuadrícula (this.x, this.y): Coordenadas discretas en el plano cartesiano del mapa donde el agente se encuentra ubicado. En la inicialización, toma la posición del altar central (mapa.baseX, mapa.baseY).

Dimensionamiento gráfico (this.tamanoCasilla): Define el tamaño en píxeles (40 px) de cada celda para escalar correctamente las coordenadas lógicas al lienzo Canvas.

```
this.x = x;
this.y = y;
this.tamanoCasilla = tamanoCasilla;
```

Imagen 1.3 declaración de la posición y el tamaño de la casilla

```
// Posicion el altar
this.baseX = Math.floor(columnas / 2);
this.baseY = Math.floor(filas / 2);
this.grid[this.baseY][this.baseX] = TIPO_CASILLA.BASE_ESPEJO;
```

Imagen 1.4 Posición del altar (base) donde el agente sale y debe regresar

Reserva de energía (this.energia): Escalar numérico que representa el nivel de batería del agente, iniciado en el 100%. Cada desplazamiento en el mapa consume 1 unidad de energía, imponiendo un límite operativo.

Tasa de recarga (this.tasaRecarga): Valor constante fijado en 20%. Define la cantidad de batería que el agente recupera por ciclo al permanecer sobre la casilla del altar.

```
this.energia = 100;
this.tasaRecarga = 20;
```

Imagen 1.5 Estado de energía y la tasa de recarga

Estado de carga (this.tieneFragmento): Variable booleana (true/false) que determina si C.C. transporta actualmente un cristal de espejo. Modifica la lógica de decisión y activa un resplandor cian sobre su sprite.

Contador de cristales (this.fragmentosRecolectados): Registro numérico acumulativo de fragmentos que han sido recolectados y entregados con éxito en la base.

```
this.tieneFragmento = false;
this.fragmentosRecolectados = 0;
```

Imagen 1.6 Estados del agente si tiene fragmento y cuantos recolectados

```
if (this.tieneFragmento) {
    ctx.fillStyle = 'rgba(56, 189, 248, 0.35)';
    ctx.fillRect(posX, posY, this.tamanoCasilla, this.tamanoCasilla);
}
```

Imagen 1.7 En caso de estar cargando un cristal se ilumina el agente

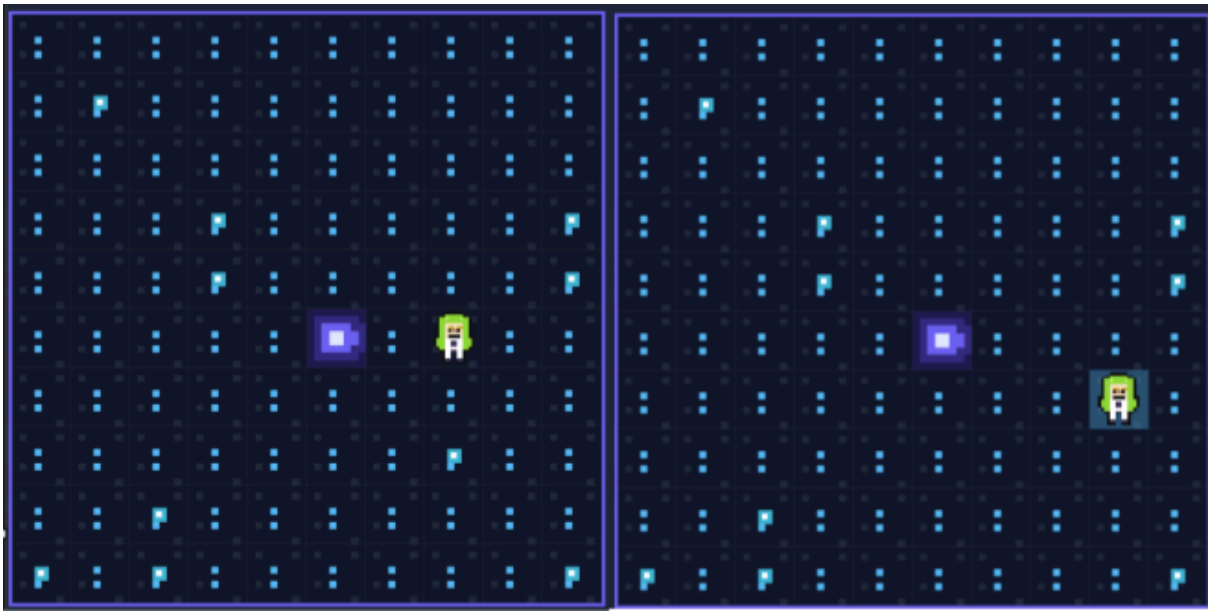


Imagen 1.8 Representación gráfica donde se puede observar cuando no tiene un cristal y al cargarlo

Descriptor de acción (this.estado): Cadena de texto que describe la actividad en curso (por ejemplo, "Dormida", "Búsqueda", o "RECARGAR") utilizada para actualizar la interfaz gráfica en tiempo real.

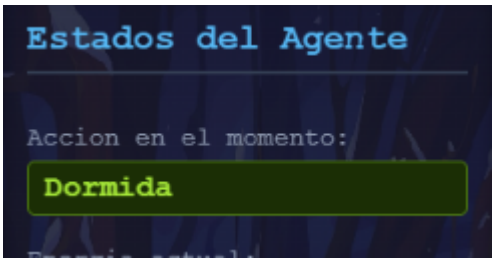


Imagen 1.9 Estado actual del agente, se actualiza en tiempo real dependiendo de la accion que está realizando el agente

Modelado y Generación del Entorno

Estructura Matricial del Mundo

El entorno en el que opera el agente se define mediante la clase Mapa, la cual encapsula la representación espacial del mundo como una grilla o matriz bidimensional.



Imagen 1.10 Clase que contiene los elementos del mapa

Enumeración de Tipos de Casilla (TIPO_CASILLA): Se utiliza un objeto constante para definir numéricamente el contenido de cada celda: VACIA (0), BASE_ESPEJO (1) y FRAGMENTO (2).

Construcción de la Grilla: Mediante el constructor, se genera una matriz de 10 x 10 (por defecto) usando arreglos anidados (Array(filas).fill(null).map(...)), donde cada casilla se inicializa en estado totalmente vacío (TIPO_CASILLA.VACIA).

```
export class Mapa {
    constructor(columnas = 10, filas = 10, tamanoCasilla = 40) {
        this.columnas = columnas;
        this.filas = filas;
        this.tamanoCasilla = tamanoCasilla;

        // Creacion de una cuadricula limpia sin muros
        this.grid = Array(filas).fill(null).map(() => Array(columnas).fill(TIPO_CASILLA.VACIA));
```

Imagen 2.0 Datos de la matriz

Posicionamiento del Altar y Fragmentos

El mapa administra la ubicación inicial del objetivo principal (el altar o base) y la dispersión aleatoria de los fragmentos del espejo:

Ubicación Central del Altar: La base se calcula automáticamente en el centro geométrico de la matriz mediante Math.floor(columnas / 2) y Math.floor(filas / 2). Esta coordenada (baseX, baseY) se asigna como TIPO_CASILLA.BASE_ESPEJO y sirve como punto de reaparición, recarga e integración de fragmentos para C.C.

Generación Aleatoria de Cristales (generarFragmentos): Método que distribuye 10 fragmentos de espejo en el mapa. Utiliza un bucle while que selecciona coordenadas (x, y) aleatorias, validando que la celda de destino contenga estrictamente TIPO_CASILLA.VACIA para evitar sobrescribir el altar o empalmar fragmentos en una misma posición.



```
        // Posicion el altar
        this.baseX = Math.floor(columnas / 2);
        this.baseY = Math.floor(filas / 2);
        this.grid[this.baseY][this.baseX] = TIPO_CASILLA.BASE_ESPEJO;

        this.generarFragmentos(10);
    }

    generarFragmentos(cantidad) {
        let colocados = 0;
        while (colocados < cantidad) {
            const x = Math.floor(Math.random() * this.columnas);
            const y = Math.floor(Math.random() * this.filas);

            // Asegurar que solo se coloquen en casillas totalmente vacías
            if (this.grid[y][x] === TIPO_CASILLA.VACIA) {
                this.grid[y][x] = TIPO_CASILLA.FRAGMENTO;
                colocados++;
            }
        }
    }
```

Imagen 2.1 Generación de la casilla donde está el altar y fragmentos del espejo dispersos por el mapa

Renderizado Pixel-Art por Texturas

La visualización del mapa no emplea imágenes externas, sino un sistema dinámico de dibujo píxel por píxel sobre la grilla Canvas.

Plantillas de Mosaicos (8 x 8 Píxeles): Cada celda se divide conceptualmente en una sub-grilla de 8 x 8 micro-píxeles. Se definen matrices de caracteres para los tipos de terreno:

- TILE_HIELO: Modela el suelo congelado con variaciones de tono.
- TILE_ALTAR: Define el grabado de piedra y cristal central del altar.

Colores del mapa (PALETA_MAPA): Asocia cada carácter de las plantillas a un código hexadecimal específico (por ejemplo, 'b' para azul noche de fondo, 'C' para el brillo del hielo o 'E' para el marco del altar).



```
// Textura de Nieve / Hielo despejado (8x8 píxeles)
const TILE_HIELO = [
    "bbbbbbbb",
    "bBbbbbBb",
    "bbbbbbbb",
    "bbbCbbbb",
    "bbbbbbbb",
    "bBbCbbbb",
    "bbbbbbbb",
    "bbbbbbBb"
];

// Textura del Altar del Espejo Central (8x8 píxeles)
const TILE_ALTAR = [
    "kkkkkkkk",
    "kKKKKKKk",
    "kKEEEEKK",
    "kKEhhEEK",
    "kKEhhEEK",
    "kKEEEEKK",
    "kKKKKKKk",
    "kkkkkkkk"
];

const PALETA_MAPA = {
    'b': '#0f172a', // Azul noche (suelo)
    'B': '#1e293b', // Detalle nieve
    'C': '#38bdf8', // Brillo de hielo
    'k': '#1e1b4b', // Borde altar
    'K': '#312e81', // Piedra
    'E': '#6366f1', // Marco
    'h': '#e0e7ff'  // Cristal central
};
```

Imagen 2.2 Paleta de colores y matriz para el dibujo

Bucle de Dibujado (dibujar):

- Determina el tamaño de cada sub-píxel dividiendo el tamaño de la casilla entre 8 (tamPixel = tamanoCasilla / 8).
- Recorre la grilla principal y sub-recorre la plantilla de 8×8 para pintar los rectángulos individuales mediante ctx.fillRect().
- Si la casilla contiene un FRAGMENTO, superpone un rombo/cuadrado cian con un brillo blanco interno.
- Dibuja una cuadrícula fina semitransparente (rgba(255, 255, 255, 0.03)) mediante ctx.strokeRect() para delimitar visiblemente cada casilla del mapa.



```
    dibujar(ctx) {
        const tamPixel = this.tamanoCasilla / 8;

        for (let r = 0; r < this.filas; r++) {
            for (let c = 0; c < this.columnas; c++) {
                const posX = c * this.tamanoCasilla;
                const posY = r * this.tamanoCasilla;
                const tipo = this.grid[r][c];

                // Textura del suelo o altar
                const plantilla = (tipo === TIPO_CASILLA.BASE_ESPEJO) ? TILE_ALTAR : TILE_HIELO;

                for (let px = 0; px < 8; px++) {
                    for (let py = 0; py < 8; py++) {
                        const charColor = plantilla[py][px];
                        ctx.fillStyle = PALETA_MAPA[charColor];
                        ctx.fillRect(posX + (px * tamPixel), posY + (py * tamPixel), tamPixel, tamPixel);
                    }
                }

                // Fragmento de espejo
                if (tipo === TIPO_CASILLA.FRAGMENTO) {
                    ctx.fillStyle = '■#06b6d4';
                    ctx.fillRect(posX + 15, posY + 15, 10, 10);
                    ctx.fillStyle = '■#ffffff';
                    ctx.fillRect(posX + 18, posY + 18, 4, 4);
                }

                ctx.strokeStyle = '□rgba(255, 255, 255, 0.03)';
                ctx.strokeRect(posX, posY, this.tamanoCasilla, this.tamanoCasilla);
            }
        }
    }
}
```

Imagen 2.3 Método para el dibujado

Ciclo de Decisiones: Percepción, Regla de Reflejo y Acción

Percepción Inmediata

El método percibir(mapa) actúa como la interfaz sensorial primaria del agente. Siguiendo el principio de un Agente de Reflejo Simple, el escaneo se limita exclusivamente al plano espacial instantáneo donde C.C. está parada.

Consulta matricial: Accede a mapa.grid[this.y][this.x] para identificar la casilla actual.

Objeto de percepción: Retorna un paquete de datos con variables booleanas:

- **hayFragmento:** Determina si en la posición actual existe un cristal de espejo (TIPO_CASILLA.FRAGMENTO).

- **enBase:** Identifica si C.C. se encuentra exactamente sobre el altar central (TIPO_CASILLA.BASE_ESPEJO).
- **energiaIncompleta:** Evalúa si la batería se encuentra por debajo de su límite máximo (this.energia < 100).

```
percibir(mapa) {
    const casillaActual = mapa.grid[this.y][this.x];
    return {
        hayFragmento: casillaActual === TIPO_CASILLA.FRAGMENTO,
        enBase: casillaActual === TIPO_CASILLA.BASE_ESPEJO,
        energiaIncompleta: this.energia < 100
    };
}
```

Imagen 2.4 Método donde el agente percibe el mapa

Tabla Jerárquica de Condición-Acción

El método reglaReflejo(percepcion) encapsula la función de decisión del agente. Aplica una tabla condicional priorizada mediante estructuras if, asegurando que las necesidades operativas críticas se ejecuten antes que el movimiento.

- **Prioridad 1 — Agotamiento Batería (SIN_ENERGIA):** Si this.energia <= 0, interrumpe cualquier operación posterior.
- **Prioridad 2 — Recolección (RECOGER_FRAGMENTO):** Si detecta un fragmento en su casilla (percepcion.hayFragmento) y no lleva uno cargado (!this.tieneFragmento).
- **Prioridad 3 — Depósito (DEPOSITAR_FRAGMENTO):** Si está sobre la base (percepcion.enBase) y transporta un cristal (this.tieneFragmento).
- **Prioridad 4 — Recarga (RECARGAR):** Si se encuentra en el altar (percepcion.enBase) y su batería no está completa (percepcion.energiaIncompleta).
- **Prioridad 5 — Exploración Libre (MOVER_ALEATORIO):** Si no se cumple ninguna de las condiciones anteriores, la regla por defecto es continuar el desplazamiento no dirigido.

```
reglaReflejo(percepcion) {
    if (this.energia <= 0) {
    return 'SIN_ENERGIA';
}
    if (percepcion.hayFragmento && !this.tieneFragmento) {
        return 'RECOGER_FRAGMENTO';
    }
    if (percepcion.enBase && this.tieneFragmento) {
        return 'DEPOSITAR_FRAGMENTO';
    }
    if (percepcion.enBase && percepcion.energiaIncompleta) {
        return 'RECARGAR';
    }
    return 'MOVER_ALEATORIO';
}
```

Imagen 2.5 método percepción en el cual el agente basa sus decisiones

Módulo de Actuar

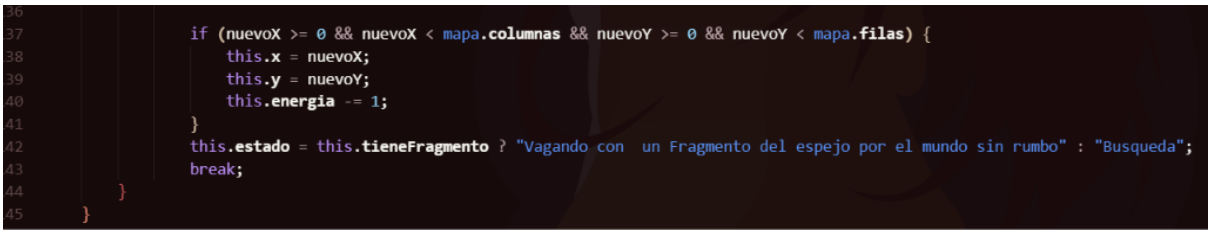
El método actuar(accion, mapa) transforma la decisión simbólica retornada por la regla de reflejo en cambios concretos en el entorno y en las variables internas del agente.

- **SIN_ENERGIA:** Modifica la propiedad this.estado declarando que C.C. ha entrado en un letargo permanente, deteniendo su interacción física.
- **RECOGER_FRAGMENTO:** Asigna this.tieneFragmento = true y sobrescribe la celda en el mapa cambiando TIPO_CASILLA.FRAGMENTO a TIPO_CASILLA.VACIA.
- **DEPOSITAR_FRAGMENTO:** Libera la carga (this.tieneFragmento = false) e incrementa en 1 el contador global this.fragmentosRecolectados.
- **RECARGAR:** Incrementa el valor de this.energia sumando la tasa de recarga ($+ 20$), utilizando Math.min() para asegurar que el tope nunca exceda del 100%.
- **MOVER_ALEATORIO:**
    1. Define los desplazamientos ortogonales posibles (Arriba: {dx: 0, dy: -1}, Abajo: {dx: 0, dy: 1}, Izquierda: {dx: -1, dy: 0}, Derecha: {dx: 1, dy: 0}).
    2. Selecciona un vector de dirección al azar mediante Math.random().
    3. Calcula las nuevas coordenadas candidatas (nuevoX, nuevoY) y valida que estén dentro de los límites permitidos de la matriz (0 <= nuevoX < mapa.columnas y 0 <= nuevoY < mapa.filas).
    4. Si la celda es válida, actualiza las coordenadas de C.C. y descuenta 1 unidad de energía (this.energia -= 1).



```
99      // Acciones que realiza el agente
100     actuar(accion, mapa) {
101         switch (accion) {
102             case 'SIN_ENERGIA':
103             this.estado = "la Agente t0 C.C. ha entrado en un letargo eterno.";
104             console.log("la Agente t0 C.C.ha entrado en un letargo eterno.");
105             break;
106
107             case 'RECOGER_FRAGMENTO':
108                 this.tieneFragmento = true;
109                 mapa.grid[this.y][this.x] = TIPO_CASILLA.VACIA;
110                 console.log("C.C. recupero un fragmento de espejo.");
111                 break;
112
113             case 'DEPOSITAR_FRAGMENTO':
114                 this.tieneFragmento = false;
115                 this.fragmentosRecolectados++;
116                 console.log(`C.C. deposito un fragmento en el altar. Total: ${this.fragmentosRecolectados}`);
117                 break;
118
119             case 'RECARGAR':
120                 this.energia = Math.min(100, this.energia + this.tasaRecarga);
121                 this.estado = `La agente C.C. esta recuperando energia en el altar. (${this.energia}%)`;
122                 console.log(`La agente C.C. esta recuperando energia en el altar. Energia actual: ${this.energia}%`);
123                 break;
124
125             case 'MOVER_ALEATORIO':
126                 const direcciones = [
127                     { dx: 0, dy: -1 },
128                     { dx: 0, dy: 1 },
129                     { dx: -1, dy: 0 },
130                     { dx: 1, dy: 0 }
131                 ];
132                 const d = direcciones[Math.floor(Math.random() * direcciones.length)];
133
134                 const nuevoX = this.x + d.dx;
135                 const nuevoY = this.y + d.dy;
```

Imagen 2.6 Acciones que realiza el agente



```
36
37                 if (nuevoX >= 0 && nuevoX < mapa.columnas && nuevoY >= 0 && nuevoY < mapa.filas) {
38                     this.x = nuevoX;
39                     this.y = nuevoY;
40                     this.energia -= 1;
41                 }
42                 this.estado = this.tieneFragmento ? "Vagando con  un Fragmento del espejo por el mundo sin rumbo" : "Busqueda";
43                 break;
44         }
45     }
```

Imagen 2.6.1 Continuación del código de la imagen 2.6

Representación Gráfica del Agente

Mapeo del Sprite de C.C.

El aspecto visual del agente no depende de imágenes GIF o PNG externas, sino que se construye en tiempo de ejecución mediante arte en píxeles (pixel-art) vectorial programado directamente en la clase AgenteCC:

Matriz de Caracteres (SPRITE_CC): Corresponde a una cuadrícula textual de 18 filas por 16 columnas donde cada símbolo representa la presencia de un elemento o color en la figura del personaje.

Mapeo Cromático (PALETA): Diccionario de claves-valores que vincula cada carácter de la matriz con su respectivo código hexadecimal:

- 'K' (#121212): Delineado exterior de la figura.
- 'G' (#8cd838) y 'g' (#b4f05c): Tono base y brillos del cabello característico de C.C.
- 'S' (#fbe2c3), 'E' (#cca028) y 'M' (#b88168): Tono de piel, ojos dorados y sombras del rostro.
- 'W' (#ffffff) y 'D' (#3b3154): Traje blanco con detalles morados.
- '.' (null): Representa zonas transparentes sin renderizado de píxel.

Cálculo de Centrado y Escala:

1. Define tamPixel = 2, haciendo que la figura final mida 32×36 píxeles reales.
2. Para mantener a C.C. perfectamente alineada dentro de la casilla de 40×40 píxeles, calcula un desplazamiento dinámico:

$$offsetX = posX + \frac{tamanoCasilla - (cols \cdot tamPixel)}{2}$$

$$offsetY = posY + \frac{tamanoCasilla - (filas \cdot tamPixel)}{2}$$

Mediante un doble bucle for, recorre la matriz y dibuja cada celda no nula usando ctx.fillRect().

```
//  de colores para C.C (Agente)
const PALETA = {
    '.': null,                  // Transparente
    'K': '#121212',             // Delineado negro
    'G': '#8cd838',             // Pelo verde C.C.
    'g': '#b4f05c',             // Brillo de pelo
    'S': '#fbe2c3',             // Piel
    'E': '#cca028',             // Ojos dorados
    'M': '#b88168',             // Sombra rostro / boca
    'W': '#ffffff',             // Traje blanco
    'D': '#3b3154'              // Detalles traje morado/rojo
};
```

```
// Matriz para dibujar al agente
const SPRITE_CC = [
    ".....KKKKKK.....",
    "...KKGGGGGGKK...",
    "..KGGgGGGGgGGK..",
    "..KGGGGGGGGGGK..",
    "..KGGSSSSSSGGK..",
    "..KGGSEESEEGGK..",
    "..KGGSKMMKSGGK..",
    "..KGGSSSSSSGGK..",
    ".KKGGSKKKKSGGKK.",
    ".KGGGWWWWWGGGK.",
    ".KGGGWWDDWWGGGK.",
    ".KGGGWWDDWWGGGK.",
    "..KGGWWWWWWGGK..",
    "...KKWWDDWWKK...",
    "....KWW..WWK....",
    "....KWW..WWK....",
    "....KDD..DDK....",
    "....KKK..KKK...."
];
```

Imagen 2.7 paleta de colores y matriz para el dibujado del agente

```
// Calcula el tamaño de cada "píxel" dentro de la casilla de 40px
const tamPixel = 2;

// Centrado del personaje en la casilla
const offsetX = posX + (this.tamanoCasilla - (cols * tamPixel)) / 2;
const offsetY = posY + (this.tamanoCasilla - (filas * tamPixel)) / 2;
```

Imagen 2.8 codigo para centrar el Agente en la casilla

Retroalimentación Visual de Carga

Para proporcionar un indicador visual inmediato al usuario sobre el estado interno del agente sin recurrir a texto flotante, el método dibujar(ctx) incluye un efecto de resplandor dinámico:

- **Detección de Carga:** Evalúa la variable booleana this.tieneFragmento antes de iterar la matriz del sprite.
- **Capa de Proyección Cyan:** Si la condición se cumple, se aplica un color de relleno semitransparente (rgba(56, 189, 248, 0.35)) mediante ctx.fillStyle y se dibuja un cuadrado completo que cubre la casilla actual de 40×40 píxeles.
- **Superposición:** Esta luz azul traslúcida se proyecta directamente sobre la textura del terreno antes de dibujar a C.C., logrando que el personaje aparezca iluminado por el fragmento que transporta.

Bucle de Simulación e Interfaz

Bucle de Ejecución Asíncrona

El archivo main.js actúa como el punto de entrada y coordinador de la simulación, unificando los módulos de lógica (AgenteCC) y entorno (Mapa) sobre un lienzo Canvas de 400×400 píxeles:

Configuración del Contexto de Renderizado: Al cargar la página (window.addEventListener('load', ...)), se obtiene el contexto 2D del canvas y se deshabilita el suavizado de imágenes (ctx.imageSmoothingEnabled = false) para preservar la nitidez visual del estilo pixel-art.
Bucle por Temporizador (setInterval): La simulación no utiliza un bucle continuo por fotogramas (requestAnimationFrame), sino un ciclo asíncrono controlado mediante setInterval(cicloSimulacion, 500). Esto establece una cadencia discreta de 2 turnos por segundo ($500ms$ por ciclo), permitiendo observar el proceso paso a paso.
Flujo del Ciclo de Simulación (cicloSimulacion): En cada tick del reloj se ejecutan metódicamente tres fases:

1. **Percepción:** El agente consulta la casilla donde se encuentra mediante agente.percibir(mapa).

2. **Toma de Decisión y Acción:** Se evalúa la regla de reflejo agente.reglaReflejo(percepcion) y se aplica el cambio en el mapa con agente.actuar(accion, mapa).
3. **Re-renderizado:** Se limpia el lienzo con ctx.clearRect(0, 0, canvas.width, canvas.height) y se vuelven a dibujar el mapa, el agente y la interfaz gráfica

```
    function cicloSimulacion() {
        const percepcion = agente.percibir(mapa);
        const accion = agente.reglaReflejo(percepcion);
        agente.actuar(accion, mapa);

        renderizar();
    }

    function iniciar() {
    if (!simulacionInterval) {
        if (agente.estado === "Dormida") {
            agente.estado = "Despertando... del largo sueño";
            renderizar();
        }

        simulacionInterval = setInterval(cicloSimulacion, 500);
        btnIniciar.disabled = true;
        btnDetener.disabled = false;
    }
}

function detener() {
    if (simulacionInterval) {
        clearInterval(simulacionInterval);
        simulacionInterval = null;


        agente.estado = "Tomando un descanso";
        renderizar();

        btnIniciar.disabled = false;
        btnDetener.disabled = true;
    }
}
```

Imagen 2.9 Función para la simulación, inicio y detenido

Sincronización de la Interfaz de Usuario (UI)

Para monitorear el comportamiento del agente sin depender exclusivamente de la consola del navegador, la función actualizarInterfazUI() vincula las variables del modelo con los elementos del DOM definidos en index.html:

- **Visualización de Estados y Atributos:** Se actualizan en tiempo real los elementos textuales con el estado actual (#txt-estado), el conteo de cristales entregados (#txt-fragmentos) y el indicador booleano de transporte de carga (#txt-cargando).
- **Barra de Energía Dinámica (#progreso-energia):** Ajusta el ancho porcentual (style.width) según el nivel numérico de agente.energia. Adicionalmente, aplica un esquema cromático condicional para reflejar el estado crítico de la batería:
    - **Verde (#22c55e):** Nivel de energía superior al 50%.
    - **Amarillo (#eab308):** Nivel de energía entre 21% y 50%.
    - **Rojo (#ef4444):** Nivel de energía igual o inferior al 20%.

```
    <div class="panel-estado">
        <h2>Estados del Agente</h2>

        <div class="dato">
            <label> Accion en el momento:</label>
            <span id="txt-estado" class="tag-estado">Busqueda</span>
        </div>

        <div class="dato">
            <label>Energia actual:</label>
            <span id="txt-energia">100%</span>
            <div class="barra-energia">
                <div id="progreso-energia" style="width: 100%;"></div>
            </div>
        </div>

        <div class="dato">
            <label>Fragmentos Integrados:</label>
            <span id="txt-fragmentos">0 / 5</span>
        </div>

        <div class="dato">
            <label>Cargando Fragmento:</label>
            <span id="txt-cargando">No</span>
        </div>
    </div>
</div>
```

Imagen 2.10 Panel donde se muestran los estados del agente en tiempo real

```
function actualizarInterfazUI() {
    txtEstado.textContent = agente.estado;
    txtEnergia.textContent = `${agente.energia}%`;
    progresoEnergia.style.width = `${agente.energia}%`;

    if (agente.energia > 50) {
        progresoEnergia.style.backgroundColor = '#22c55e';
    } else if (agente.energia > 20) {
        progresoEnergia.style.backgroundColor = '#eab308';
    } else {
        progresoEnergia.style.backgroundColor = '#ef4444';
    }

    txtFragmentos.textContent = `${agente.fragmentosRecolectados} / 10`;
    txtCargando.textContent = agente.tieneFragmento ? "Sí" : "No";
}
```

Imagen 3.0 Función donde se actualiza los colores del nivel de energía y otros elementos de la interfaz del agente

Control de Inicio y detenido de la Simulación

La interacción y control del usuario sobre la ejecución se gestiona mediante eventos asignados a los botones de control (#btn-iniciar y #btn-detener):

- **Inicio (iniciar):** Al presionar "Empezar", se activa el intervalo setInterval, se deshabilita el botón de inicio (disabled = true) y se habilita el botón de paro (disabled = false). Si el agente se encontraba en estado "Dormida", actualiza su descriptor a "Despertando... del largo sueño" antes de comenzar la exploración.
- **Detención (detener):** Al presionar "Detener", se limpia el intervalo activo (clearInterval), se restituyen los estados interactivos de los botones y se asigna el estado "Tomando un descanso" a C.C..
- **Estilo de los botones (styles.css):** La hoja de estilos define una estética temática mediante tipografía monospaciada (Courier New), paleta oscura, bordes resaltados para el Canvas, fondos graduados con imágenes personalizadas (snowy-mountains.jpg, snowy-forest.png) y animaciones con retroalimentación en estados :hover y :disabled.

```
    <div class="contenedor-botones">
        <button id="btn-iniciar" class="btn btn-iniciar">Empezar</button>
        <button id="btn-detener" class="btn btn-detener" disabled>Detener</button>
    </div>
</div>
```

Imagen 3.1 Contenedor de los botones de Iniciar y detener



```
59      function iniciar() {
60      if (!simulacionInterval) {
61          if (agente.estado === "Dormida") {
62              agente.estado = "Despertando... del largo sueño";
63              renderizar();
64          }
65
66          simulacionInterval = setInterval(cicloSimulacion, 500);
67          btnIniciar.disabled = true;
68          btnDetener.disabled = false;
69      }
70  }
71
72  function detener() {
73      if (simulacionInterval) {
74          clearInterval(simulacionInterval);
75          simulacionInterval = null;
76
77
78          agente.estado = "Tomando un descanso";
79          renderizar();
80
81          btnIniciar.disabled = false;
82          btnDetener.disabled = true;
83      }
84  }
```

Imagen 3.2 Acciones de los botones y estados del agente al utilizarlos

```css
.contenedor-botones {
    display: flex;
    gap: 10px;
    margin-top: 10px;
}

.btn {
    flex: 1;
    padding: 8px 10px;
    font-family: inherit;
    font-weight: bold;
    font-size: 0.85rem;
    text-transform: uppercase;
    border-radius: 6px;
    cursor: pointer;
    border: 2px solid transparent;
    transition: all 0.2s ease;
}

/* Boton iniciar color */
.btn-iniciar {
    background: linear-gradient(135deg, #14532d 0%, #15803d 100%);
    color: #86efac;
    border-color: #22c55e;
    box-shadow: 0 4px 12px rgba(34, 197, 94, 0.2);
}

.btn-iniciar:hover:not(:disabled) {
    background: linear-gradient(135deg, #15803d 0%, #16a34a 100%);
    color: #ffffff;
    box-shadow: 0 0 15px rgba(34, 197, 94, 0.6);
    transform: translateY(-2px);
}
```

Imagen 3.3 Estilo del contenedor y botón iniciar

```css
/* Boton detener colores */
.btn-detener {
    background: linear-gradient(135deg, #7f1d1d 0%, #b91c1c 100%);
    color: #fca5a5;
    border-color: #ef4444;
    box-shadow: 0 4px 12px rgba(239, 68, 68, 0.2);
}

.btn-detener:hover:not(:disabled) {
    background: linear-gradient(135deg, #b91c1c 0%, #dc2626 100%);
    color: #ffffff;
    box-shadow: 0 0 15px rgba(239, 68, 68, 0.6);
    transform: translateY(-2px);
}
```

Imagen 3.3.1 Continuación del estilo código anterior incluyendo el estilo del botón detener

Arquitectura agente t-1

Proyecto: C.C.'s Journey — IA Inteligencia Artificial

Repositorio: https://github.com/khwarenah/IA-Inteligencia_Artificial

Resumen

Se modificaron 21 archivos: 1,909 líneas agregadas y 191 eliminadas. main contiene el agente reflejo simple que se mueve de forma aleatoria. combinacion3 agrega memoria, los 5 sentidos, planificación con A*, salud y respawn, y nuevas entidades (dragón, aliada, brujo, trampa, castillo y árboles) sobre un mapa fijo de 25×10.

Archivos nuevos

Archivo	Descripción
astar.js	A* propio con función de costo inyectada. Heurística Manhattan × 0.5 (admisible). Devuelve la ruta sin incluir el origen.
dragon.js	Dragón enemigo de 2×2 casillas.
aliada.js	Aliada que ayuda a C.C. y ataca enemigos.
brujo.js	Brujo enemigo que drena energía y salud.
trampa.js	Trampa de hielo oculta (20 de daño).
castillo.js	Castillo decorativo que bloquea 5 casillas.
arbol.js	Árboles que funcionan como obstáculos.
pathfinding.js	Envoltorio de easystarjs. No se importa en ningún archivo (código muerto).
Sprites (.png)	dragon_sprite, aliada_sprite, brujo_sprite, castillo_sprite, arbol_cold, arbol_night.

agente.js

Estado nuevo en el constructor

- Salud (salud), vidas perdidas, estaMuerta y cayendoHielo.
- Memoria: memoriaBase, posAnterior y mapaMental.
- Obstáculos: obstaculosBloqueantes.
- Plan A*: rutaPlanificada, objetivoPlanificado y objetivoCristalFijo.

celdasPeligroVivo y sentidosActuales

Los 5 sentidos del Agente

Sentido	Función
Vista	Cuadrante 3×3 alrededor de C.C. Detecta al dragón y registra su zona de peligro.
Oído	Distancia Manhattan al altar con niveles FUERTE, MEDIO y DÉBIL.
Olfato	Localiza el cristal más cercano y calcula la intensidad.
Tacto	Costo del terreno: normal 0.5, hielo 2, nieve 4.
Gusto	Detecta si está en el altar y cuánta energía puede recargar.

Vista

El método percibirVista(mapa, dragon) implementa la lógica de Exploración de Vecindad de Moore (Matriz 3 x 3). A diferencia del T0 (que solo miraba su propia casilla), este método le otorga a C.C. un rango de visión de 1 casilla alrededor en todas las direcciones.

Generación de la Matriz 3 x 3

Los dos bucles for anidados iteran sobre los desplazamientos relativos (dx, dy) en el rango $[-1, 1]$:

$dy \in \{-1, 0, 1\}, dx \in \{-1, 0, 1\}$

Esto genera las 9 combinaciones posibles relativas a la posición del agente.

Transformación a Coordenadas Absolutas

Para mapear la visión al mapa real, calcula la posición absoluta de cada celda visible:

$$nx = x + dx$$

$$ny = y + dy$$

Donde (x, y) es la ubicación actual de C.C. y (nx, ny) es la celda adyacente observada.

Validación de Fronteras del Mapa

Para evitar errores de índice fuera de rango (IndexOutOfBounds) al estar en los bordes o esquinas del mapa, valida que las coordenadas evaluadas estén dentro de los límites de la matriz:

$$0 \leq nx < mapa.columnas \wedge 0 \leq ny < mapa.filas$$

Inspección de Contenido y Detección del Dragón

Para cada casilla válida observada:

- **mapa.grid[ny][nx]**: Lee el terreno o elemento presente (vacío, base o cristal).
- **dragon?.ocupaCasilla?.(nx, ny)**: Comprueba de forma segura (usando *optional chaining* ?.) si alguna parte del cuerpo del Dragón se encuentra sobre la posición (nx, ny).

Marcado de Peligro

Si la condición detectaDragon es verdadera (true):

if (detectaDragon) => registrarZonaPeligro(nx,ny,1)

Suma la casilla (nx, ny) a su capa de alerta o memoria de peligro, permitiéndole al agente aplicar restricciones en su toma de decisiones (aumenta la penalización de costo para evitar dirigirse a esa celda).

```
percibirVista(mapa, dragon) {
    const observacion = [];
    for (let dy = -1; dy <= 1; dy++) {
        for (let dx = -1; dx <= 1; dx++) {
            const nx = this.x + dx, ny = this.y + dy;
            if (nx >= 0 && nx < mapa.columnas && ny >= 0 && ny < mapa.filas) {
                const detectaDragon = dragon?.ocupaCasilla?.(nx, ny) || false;
                observacion.push({ x: nx, y: ny, contenido: mapa.grid[ny][nx], hayDragon: detectaDragon });
                if (detectaDragon) this.registrarZonaPeligro(nx, ny, 1);
            }
        }
    }
    return observacion;
}
```

Imagen 3.4 Método para percibir vista

Oído

El método **percibirOido(mapa)** simula un sensor de orientación basado en **resonancia acústica/mágica** hacia el Altar Base. Permite que C.C. reconozca qué tan cerca está de su punto de recarga sin necesidad de tener visión directa.

Cálculo Matemático: Distancia Manhattan

El núcleo del método calcula la distancia ortogonal en cuadrícula entre la posición actual del agente (x, y) y el altar base (x_{base}, y_{base}) mediante la **Distancia Manhattan**:

$$d = |x - x_{base}| + |y - y_{base}|$$

const d = Math.abs(this.x - baseX) + Math.abs(this.y - baseY);

Mide el número mínimo de pasos horizontales y verticales requeridos para llegar al altar en una grilla discreta donde no existen desplazamientos diagonales.

```
percibirOido(mapa) {
    const baseX = mapa?.baseX ?? this.memoriaBase.x;
    const baseY = mapa?.baseY ?? this.memoriaBase.y;
    const d = Math.abs(this.x - baseX) + Math.abs(this.y - baseY);
    const nivel = (d <= 2 && 'FUERTE') || (d <= 5 && 'MEDIO') || 'DEBIL';
    return { nivel, distancia: d, descripcion: `Resonancia ${nivel} (${d} casillas del Altar)` };
}
```

Imagen 3.5 Método para el sentido del oído

Resolución Robusta de Coordenadas

const baseX = mapa?.baseX ?? this.memoriaBase.x;

const baseY = mapa?.baseY ?? this.memoriaBase.y;

Usa encadenamiento opcional (?.) y el operador de fusión nula (??).

Si el objeto mapa existe y provee las coordenadas de la base, las toma directamente.

Si el mapa no está disponible en ese fotograma, recurre a la coordenada guardada en memoria (this.memoriaBase), evitando que la simulación lance un error (TypeError).

Clasificación de Intensidad

const nivel = (d <= 2 && 'FUERTE') || (d <= 5 && 'MEDIO') || 'DEBIL';

Aprovecha la evaluación de cortocircuito de JavaScript para categorizar la percepción acústica en tres umbrales discretos:

1. **FUERTE**: $d \leq 2$ casillas (C.C. escucha el altar justo al lado).
2. **MEDIO**: $2 < d \leq 5$ casillas (C.C. está a distancia intermedia).
3. **DEBIL**: $d > 5$ casillas (C.C. está lejos en los confines del mapa).

Estructura del Objeto de Percepción

return { nivel, distancia: d, descripcion: `Resonancia $nivel$({d} casillas del Altar)` };

Retorna un paquete con el nivel categórico, la métrica numérica precisa y una descripción formateada para la interfaz de usuario o la consola.

Olfato

El método percibirOlfato(mapa) simula el sentido del olfato del agente C.C. Su objetivo es calcular la intensidad de proximidad (gradiente inverso de distancia) respecto al fragmento de cristal más cercano en todo el mapa.

Escaneo del Mapa y Filtrado de Cristales

const grid = mapa?.grid || [];

const fragmentos = [];

grid.forEach((row, r) => row.forEach((val, c) => { if (val === TIPO_CASILLA.FRAGMENTO) fragmentos.push({ x: c, y: r }); }));

- Recorre la matriz del mapa (`grid`).
- Cada vez que encuentra una casilla con un cristal (TIPO_CASILLA.FRAGMENTO), guarda sus coordenadas $\{x, y\}$ dentro del arreglo fragmentos.

Cálculo de Distancias Manhattan

const distancias = fragmentos.map(f => Math.abs(this.x - f.x) + Math.abs(this.y - f.y));

Calcula la **Distancia Manhattan** desde la posición actual de C.C. (x, y) hacia cada uno de los fragmentos encontrados en el mapa:

d = | xc.c - xfragmento| + |yC.C - yfragmento|

Identificación del Fragmento Más Cercano

const menorDistancia = sinFragmentos ? Infinity : Math.min(...distancias);

- Si no quedan cristales en el mapa (sinFragmentos === true), asigna Infinity.
- Si existen fragmentos, usa Math.min(...distancias) para aislar la distancia hacia el objetivo más próximo ($d_{\min}$).

Fórmula Matemática de Intensidad del Olfato

const intensidad = sinFragmentos ? 0 : Number((1 / (menorDistancia + 1)).toFixed(2));

La intensidad del olor se rige por la función del Inverso de la Distancia:

$$I = \frac{1}{d_{\min} + 1}$$

- **Propósito del $+ 1$:** Evita la división entre cero $(1/0)$ cuando C.C. se encuentra exactamente sobre la casilla del cristal ($d_{\min} = 0$).
- **Escala de Valores:**
  - **Sobre el cristal ($d = 0$):** $I = \frac{1}{0+1} = 1.00$ (Intensidad máxima).
  - **A 1 casilla de distancia ($d = 1$):** $I = \frac{1}{1+1} = 0.50$.
  - **A 4 casillas de distancia ($d = 4$):** $I = \frac{1}{4+1} = 0.20$.

A medida que C.C. se aproxima al cristal objetivo, **la intensidad aumenta hacia** 1.00; si se aleja, **se degrada hacia** 0.00.

Retorno de Datos

Entrega un objeto con:

- intensidad: El valor numérico (0.00 a 1.00).
- distancia: La cantidad de casillas hasta el cristal más cercano.
- objetivo: La coordenada $\{x, y\}$ del cristal más cercano (o null si no hay).
- descripcion: Texto descriptivo para la consola o panel de estado (ejemplo: "Intensidad 0.33 (Cristal a 2 cas)").

```
percibirOlfato(mapa) {
    const grid = mapa?.grid || [];
    const fragmentos = [];
    grid.forEach((row, r) => row.forEach((val, c) => { if (val === TIPO_CASILLA.FRAGMENTO) fragmentos.push({ x: c, y: r }); }));
    const sinFragmentos = fragmentos.length === 0;
    const distancias = fragmentos.map(f => Math.abs(this.x - f.x) + Math.abs(this.y - f.y));
    const menorDistancia = sinFragmentos ? Infinity : Math.min(...distancias);
    const intensidad = sinFragmentos ? 0 : Number((1 / (menorDistancia + 1)).toFixed(2));
    return {
        intensidad, distancia: menorDistancia,
        objetivo: sinFragmentos ? null : fragmentos[distancias.indexOf(menorDistancia)],
        descripcion: sinFragmentos ? 'Sin fragmentos en el mapa' : `Intensidad ${intensidad} (Cristal a ${menorDistancia} cas)`
    };
}
```

Imagen 3.6 Método para el sentido del Olfato

Tacto

El método **`percibirTacto(mapa)`** implementa el sensor del **tacto/fricción de terreno**. Su función principal es medir la resistencia física de la casilla sobre la que está parada C.C., determinando cuánto "cuesta" energéticamente moverse por ella y ofreciendo una función para validar colisiones con los bordes del mapa.

Lectura del Terreno Actual

const casillaActual = mapa.grid[this.y][this.x];

Obtiene el identificador de la casilla actual de C.C. consultando sus coordenadas de posición (x, y) directamente en la matriz del mapa.

Evaluación de Resistencia y Fricción

let costoTerreno = 0.5, tipoNombre = "Normal";

if (casillaActual === TIPO_CASILLA.NIEVE) { costoTerreno = 4; tipoNombre = "Nieve (Alta resistencia)"; }

else if (casillaActual === TIPO_CASILLA.HIELO) { costoTerreno = 2; tipoNombre = "Hielo (Resistencia media)"; }

Asigna de forma condicional el costo de energía consumido por desplazamiento según el tipo de superficie:

- **Suelo Normal (por defecto):** Costo $= 0.5$ unidades de energía.
- **Terreno de Hielo (TIPO_CASILLA.HIELO):** Costo $= 2.0$ unidades (Resistencia media).
- **Terreno de Nieve (TIPO_CASILLA.NIEVE):** Costo $= 4.0$ unidades (Alta resistencia/dificultad de paso).

Función Integrada de Validación de Límites

puedoMovermeA: (nx, ny) => (nx >= 0 && nx < mapa.columnas && ny >= 0 && ny < mapa.filas)

Retorna una función flecha lambda que evalúa si una posición de destino (nx, ny) es válida dentro de los límites de la matriz:

$$0 \leq nx < mapa.columnas \wedge 0 \leq ny < mapa.filas$$

Esto le permite al agente comprobar si un paso candidato está libre de colisión con los muros externos del mapa antes de ejecutar la acción.

Estructura del Objeto de Retorno

Retorna un paquete de datos que contiene:

- casillaActual: El ID numérico o enum del terreno actual.
- costoMovimiento: El valor numérico del gasto de batería (0.5, 2, o 4).
- tipoNombre: Descripción en texto para la UI o consola.
- puedoMovermeA: La función auxiliar de validación espacial.

```
221     percibirTacto(mapa) {
222         const casillaActual = mapa.grid[this.y][this.x];
223         let costoTerreno = 0.5, tipoNombre = "Normal";
224         if (casillaActual === TIPO_CASILLA.NIEVE) { costoTerreno = 4; tipoNombre = "Nieve (Alta resistencia)"; }
225         else if (casillaActual === TIPO_CASILLA.HIELO) { costoTerreno = 2; tipoNombre = "Hielo (Resistencia media)"; }
226         return {
227             casillaActual, costoMovimiento: costoTerreno, tipoNombre,
228             puedoMovermeA: (nx, ny) => (nx >= 0 && nx < mapa.columnas && ny >= 0 && ny < mapa.filas)
229         };
230     }
```

Imagen 3.7 Método de percibir Tacto

Gusto

El método **`percibirGusto(mapa)`** completa la suite de los 5 sentidos del agente, representando metafóricamente la **capacidad de absorción energética** que C.C. experimenta al estar en contacto con la energía mística del Altar Base.

Validación de Presencia en el Altar

const enBase = (this.x === mapa.baseX) && (this.y === mapa.baseY);

Evalúa de forma booleana si la posición actual del agente (x, y) coincide exactamente con las coordenadas absolutas de la base central (x_{base}, y_{base}).

Coeficiente de Eficiencia

const eficiencia = enBase ? 1.0 : 0.0;

Aplica un factor multiplicador binario de absorción:

- **En la base (enBase === true):** Eficiencia del \$100\%\$ ($eficiencia = 1.0$).
- **Fuera de la base (enBase === false):** Eficiencia del \$0\%\$ ($eficiencia = 0.0$).

Cálculo de la Energía Potencial a Recargar

*const recargaPosible = Math.min(100 - this.energia, this.tasaRecarga * eficiencia);*

Calcula la cantidad exacta de unidades de batería que C.C. puede absorber en el turno actual mediante la fórmula:

$$recargaPosible = \min(100 - E_{actual}, R \cdot \eta)$$

Donde:

- E_{actual} es el nivel actual de batería (this.energia).
- R Es la tasa nominal de recarga por ciclo (this.tasaRecarga).
- η es la eficiencia del terreno (eficiencia).

Propósito de Math.min: Garantiza que la recarga se ajuste dinámicamente al margen restante para no sobrepasar el tope del 100%. Por ejemplo, si C.C. tiene 90% de energía y la tasa es $+20$, el método calcula $\min(10, 20) = 10$, recuperando solo lo necesario para llegar al 100%.

Estructura del Objeto de Retorno

Devuelve un paquete con el diagnóstico completo de la recarga:

- enBase: Indicador booleano de ubicación.
- eficiencia: Multiplicador numérico (1.0 o 0.0).
- energiaARecargar: Delta exacto de energía a recuperar ($+0$ hasta $+20$).
- descripcion: Texto temático para el panel de interfaz (ejemplo: "Sabor místico: Absorbiendo energía (+20)" o "Sabor insípido: Fuera del Altar").

```
percibirGusto(mapa) {
    const enBase = (this.x === mapa.baseX) && (this.y === mapa.baseY);
    const eficiencia = enBase ? 1.0 : 0.0;
    const recargaPosible = Math.min(100 - this.energia, this.tasaRecarga * eficiencia);
    return {
        enBase, eficiencia, energiaARecargar: recargaPosible,
        descripcion: enBase ? `Sabor místico: Absorbiendo energía (+${recargaPosible})` : 'Sabor insípido: Fuera del Altar'
    };
}
```

Imagen 3.8 Método para que el Agente perciba el gusto

Algoritmo A*

La heurística original de movimiento (`evaluarMovimiento` / `seleccionarMejorPaso`) decide un solo paso a la vez: en cada turno mira únicamente las 4 casillas vecinas, les asigna un puntaje según la memoria del agente, y elige la de menor puntaje. Es una búsqueda de un solo movimiento: funciona para explorar, pero no planifica una ruta completa ni garantiza que el camino elegido sea el más barato posible hacia un destino conocido.

A* resuelve la ruta completa de menor costo total desde el origen hasta un destino concreto, en vez de decidir "a ciegas" un paso a la vez. Mientras la heurística usada sea *admisible* (nunca sobreestime el costo real restante), A* garantiza encontrar el camino óptimo.

Implementación genérica: `astar.js`

El algoritmo se implementó una sola vez, de forma genérica, en `astar.js`, y lo reutilizan tanto C.C. como la Aliada y el Brujo

f(n) = g(n) + h(n)

- g(n): costo real acumulado desde el origen hasta n (suma de los costos de cada casilla recorrida en el camino).
- h(n): estimación heurística del costo restante hasta el objetivo.
- f(n): costo total estimado de un camino que pasa por n.

```
94          const costoPaso = costoCasilla(nx, ny);
95          if (!Number.isFinite(costoPaso)) continue;
96
97          const gTentativo = costoG.get(claveActual) + costoPaso;
98
99          if (!costoG.has(vKey) || gTentativo < costoG.get(vKey)) {
100             costoG.set(vKey, gTentativo);
101             vieneDe.set(vKey, claveActual);
102             listaAbierta.set(vKey, gTentativo + heuristica(nx, ny));
103         }
```

En cada paso, el algoritmo extrae de la lista abierta (`listaAbierta`) el nodo con menor `f(n)`, lo expande hacia sus 4 vecinos, y si lo que encuentra es el objetivo, reconstruye el camino completo siguiendo los punteros `vieneDe`.

`astar.js` no conoce el mapa real ni decide por sí mismo qué tan cara es una casilla. Recibe una función `costoCasilla(nx, ny)` como parámetro, y es quien la llama la que decide ese costo:

Este diseño es lo que permite que cada agente planifique según lo que él mismo sabe o percibe, no según una verdad absoluta del mapa.

La heurística admisible

La heurística admisible

h(n) = distancia_Manhattan(n, objetivo) × costoMinimoPosible

Con `costoMinimoPosible = 0.5` (el costo de terreno normal, el más barato que existe en el mapa). Esto garantiza que `h(n)` nunca sobreestime el costo real: moverse d casillas siempre cuesta como mínimo d `× 0.5`, sin importar si el camino real termina costando más por cruzar hielo, nieve o peligro. Esa propiedad (admisibilidad) es la que garantiza que A* encuentre el óptimo.

```
32  export function encontrarRutaAEstrella(mapa, inicio, objetivo, costoCasilla, costoMinimoPosible = 0.5) {
33      function heuristica(x, y) {
34          return (Math.abs(x - objetivo.x) + Math.abs(y - objetivo.y)) * costoMinimoPosible;
35      }
```

Cómo lo usa C.C.

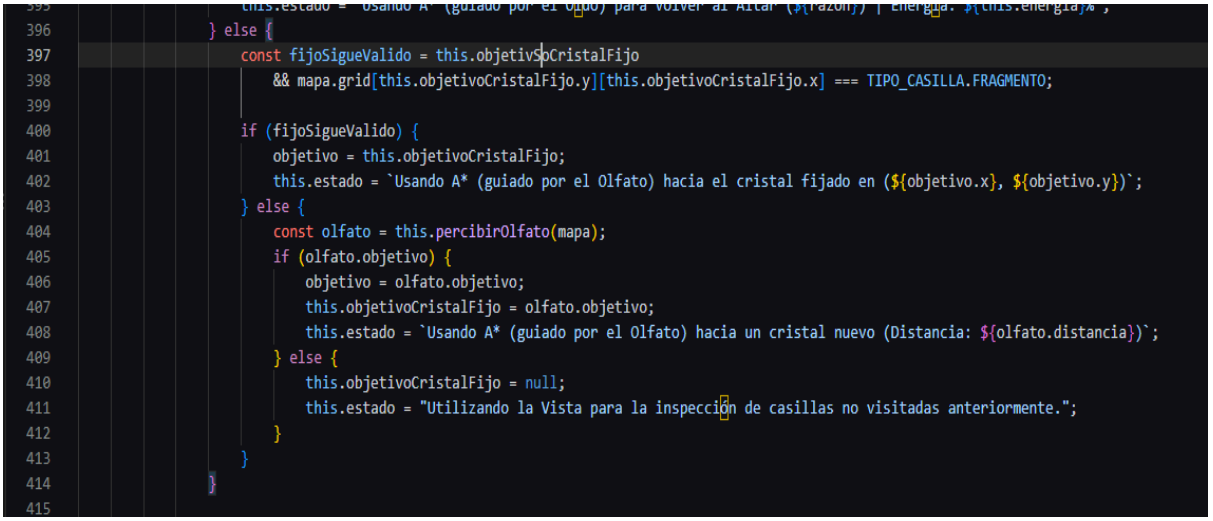
```
328     costoParaAEstrella(nx, ny) {
329         if (this.estaBloqueado(nx, ny)) return Infinity; // árbol/castillo: intransitable
330
331         const clave = `${nx},${ny}`;
332         const infoMemoria = this.mapaMental[clave];
333
334         const costoTerreno = infoMemoria?.costo ?? 0.5;
335         const peligroPermanente = infoMemoria?.esZonaFuego ? 40 : 0;
336         const peligroVivo = this.celdasPeligroVivo.has(clave) ? 40 : 0;
337
338         return costoTerreno + Math.max(peligroPermanente, peligroVivo);
339     }
340
```

C.C. planifica según lo que cree que cuesta cada casilla, no según el mapa real:

- Si ya observó una casilla (dentro de su vista de 3×3), usa su costo real (Normal = 0.5, Hielo = 2, Nieve = 4).
- Si nunca la ha visto, asume el costo más barato posible (0.5) — un supuesto optimista: lo desconocido no se trata como peligroso, se trata como "probablemente normal hasta que se demuestre lo contrario".
- Si una casilla es zona de peligro conocida (ya sea aprendida permanentemente por daño recibido, o "viva" por la posición actual del dragón/brujo), se penaliza fuertemente (+40) para que A* la evite salvo que no haya alternativa razonable.

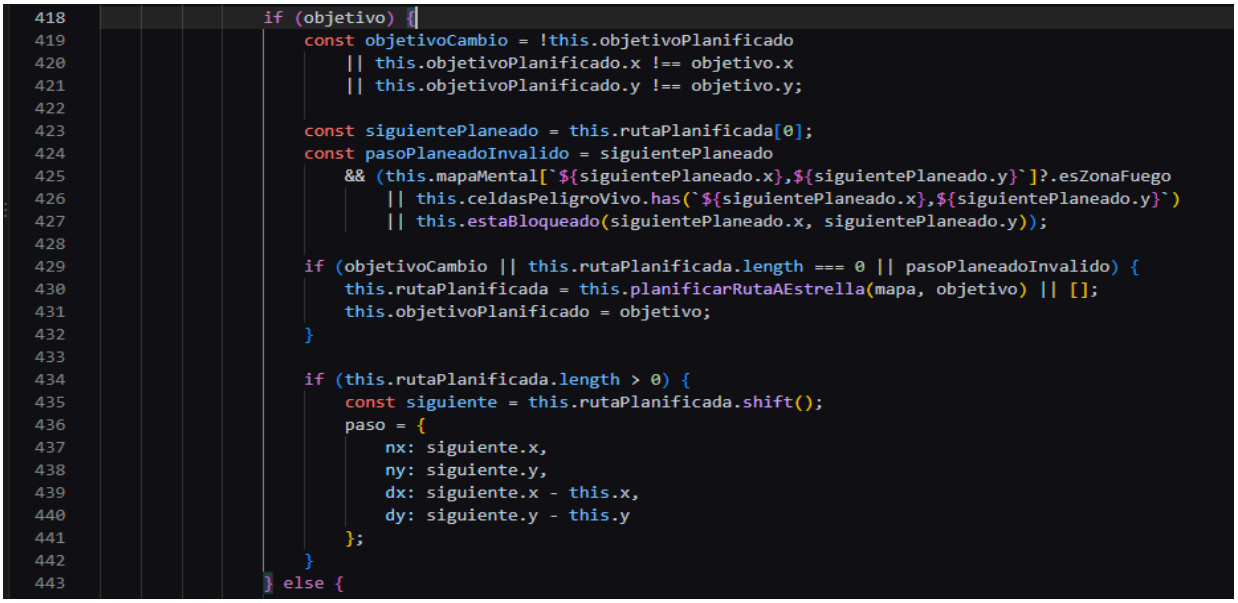
Dos mecanismos adicionales evitan que C.C. "cambie de opinión" innecesariamente:

objetivoCristalFijo: una vez que el Olfato elige un cristal como destino, C.C. se compromete con él — no lo abandona solo porque, por pura distancia, otro cristal quede momentáneamente un poco más cerca.



```
396         } else {
397             const fijoSigueValido = this.objetivoCristalFijo
398                 && mapa.grid[this.objetivoCristalFijo.y][this.objetivoCristalFijo.x] === TIPO_CASILLA.FRAGMENTO;
399
400             if (fijoSigueValido) {
401                 objetivo = this.objetivoCristalFijo;
402                 this.estado = `Usando A* (guiado por el Olfato) hacia el cristal fijado en (${objetivo.x}, ${objetivo.y})`;
403             } else {
404                 const olfato = this.percibirOlfato(mapa);
405                 if (olfato.objetivo) {
406                     objetivo = olfato.objetivo;
407                     this.objetivoCristalFijo = olfato.objetivo;
408                     this.estado = `Usando A* (guiado por el Olfato) hacia un cristal nuevo (Distancia: ${olfato.distancia})`;
409                 } else {
410                     this.objetivoCristalFijo = null;
411                     this.estado = "Utilizando la Vista para la inspección de casillas no visitadas anteriormente.";
412                 }
413             }
414         }
415
```

`rutaPlanificada` / `objetivoPlanificado`: la ruta calculada por A* se guarda y se sigue paso a paso, sin recalcular en cada turno. Solo se vuelve a llamar a A* cuando el objetivo cambió, cuando ya no quedan pasos en la ruta, o cuando el siguiente paso planeado resultó ser peligroso (por ejemplo, si el dragón se movió hacia ahí). Esto es justo lo que evita el problema, ir y venir en el mismo lugar en vez de avance real.



```
418             if (objetivo) {
419                 const objetivoCambio = !this.objetivoPlanificado
420                     || this.objetivoPlanificado.x !== objetivo.x
421                     || this.objetivoPlanificado.y !== objetivo.y;
422
423                 const siguientePlaneado = this.rutaPlanificada[0];
424                 const pasoPlaneadoInvalido = siguientePlaneado
425                     && (this.mapaMental[`${siguientePlaneado.x},${siguientePlaneado.y}`]?.esZonaFuego
426                         || this.celdasPeligroVivo.has(`${siguientePlaneado.x},${siguientePlaneado.y}`)
427                         || this.estaBloqueado(siguientePlaneado.x, siguientePlaneado.y));
428
429                 if (objetivoCambio || this.rutaPlanificada.length === 0 || pasoPlaneadoInvalido) {
430                     this.rutaPlanificada = this.planificarRutaAEstrella(mapa, objetivo) || [];
431                     this.objetivoPlanificado = objetivo;
432                 }
433
434                 if (this.rutaPlanificada.length > 0) {
435                     const siguiente = this.rutaPlanificada.shift();
436                     paso = {
437                         nx: siguiente.x,
438                         ny: siguiente.y,
439                         dx: siguiente.x - this.x,
440                         dy: siguiente.y - this.y
441                     };
442                 }
443             } else {
```

Cómo lo usan la Aliada y el Brujo

A diferencia de C.C., ni la Aliada ni el Brujo tienen percepción limitada — no hay un "campo de visión de 3×3" que restrinja lo que saben del terreno. Por eso su función de costo es más simple: lee el terreno real directamente del mapa.

```
198     costoTerrenoReal(mapa, nx, ny) {
199         if (!mapa || !mapa.grid || !mapa.grid[ny]) return 0.5;
200         const tipo = mapa.grid[ny][nx];
201         if (tipo === TIPO_CASILLA.NIEVE) return 4;
202         if (tipo === TIPO_CASILLA.HIELO) return 2;
203         return 0.5;
204     }
205
```

Como persiguen objetivos que se mueven (C.C., el Brujo o el Dragón, según el caso), no tiene sentido guardar una ruta en caché: se recalcula la ruta completa en cada turno que les corresponde moverse, y se toma solo el primer paso.

- Aliada: los tres modos de movimiento que tiene (perseguir al Brujo, perseguir al Dragón, acompañar a C.C.) usan A*, porque los tres tienen un objetivo concreto y puntual hacia el cual ir.
- Brujo: solo usa A* para `ASECHAR_AGENTE` (ir hacia C.C.). No lo usa para huir.

por ahora ninguno de los agentes tiene un algoritmo de huida o persecución definitivo, hasta ahora para la lógica de “huir” simplemente se desplazan en la dirección que más se aleje de su perseguidor, mientras que para “acechar” utiliza A* como camino más corto a su presa .

Memoria y peligro

- actualizarMemoria guarda costo, visitas, cristal y zona de fuego por casilla.
- registrarZonaPeligro marca zonas de fuego permanentes.
- actualizarZonasPeligroVivas marca radio 1 alrededor del dragón y el brujo en cada turno.

Decisión

- percibir(mapa, dragon, brujo) recibe más parámetros y devuelve la percepción de los 5 sentidos.
- reglaReflejo: se quitó MOVER_ALEATORIO; se agregaron REGRESAR_A_BASE (energía ≤ 40 o cargando cristal), EXPLORAR_INTELIGENTE, ESPERANDO_RESPAWN y SIN_ENERGIA.
- evaluarMovimiento y seleccionarMejorPaso: búsqueda voraz de respaldo cuando no hay objetivo.
- costoParaAEstrella: Infinity en obstáculos y penalización de 40 por peligro.
- planificarRutaAEstrella llama a astar.js.
- Un cristal “fijado” evita cambiar de objetivo; la ruta se replanifica si el siguiente paso es peligroso o está bloqueado.

Daño, muerte y respawn

- recibirDano y recibirDpsFuego restan salud.
- ahogarseEnHielo: 10% de probabilidad al pisar hielo.
- iniciarRespawnLento y reaparecer: 10 segundos fuera; al volver pierde el fragmento, restaura salud y energía y limpia la ruta.

Movimiento y dibujo

- La energía baja según el costo del terreno (antes −1 fijo).
- actualizarDireccion: sprites en 4 direcciones.
- Métodos nuevos de dibujo: _dibujarCaidaHielo y dibujarCampoVision.

Entidades nuevas

	Dragón	Aliada	Brujo
Salud	100	100	80
Movimiento	Aleatorio en radio 3 de su origen	Cada 2 turnos, con A*	Cada turno, con A*
Efecto	35 de daño por turno a distancia ≤ 1	25 de daño de ataque; restaura +20 energía y +20 salud a C.C.	Quita 8 de energía y 8 de salud por turno
Lógica	Patrulla y ataca	Ataca brujo/dragón a ≤1; los persigue a ≤6; si no, acompaña a C.C.	Huye de la aliada a ≤4 (compromiso de 3 turnos); acecha a C.C. a ≤6; si no, explora

Trampa: 20 de daño. Está oculta, se revela al pisarla y se marca como segura una vez gastada.

mapa.js

- De 10×10 con altar al centro y 10 fragmentos aleatorios a 25×10 con matriz fija (MAPA_ESTATICO).
- Nuevos tipos de casilla: HIELO (3) y NIEVE (4).
- Altar fijo en (12, 5). Se eliminó generarFragmentos.
- Métodos nuevos: esPasoValido y obtenerTipoTerreno.

- Texturas nuevas (suelo, hielo, nieve), paleta ampliada y fragmento dibujado como cristal afilado con aura.

main.js

- Importa las 6 clases nuevas.
- Mapa 25×10. Dragón en (22,5), aliada en (3,3), brujo en (15,7), trampa en (12,4) y castillo en (1,2).
- Unos 45 árboles colocados a mano, con bosquecito central y formas en “L” que protegen 3 cristales.
- Registro de obstáculos con registrarObstaculosBloqueantes.
- El ciclo llama a percibir, reglaReflejo y actuar, y luego a actualizar de dragón, aliada, brujo y trampa. Si C.C. está muerta, sólo anima al dragón.
- Actualiza la UI de salud y dibuja castillo, árboles, aliada, brujo, campo de visión, dragón y trampa.
- iniciar() y detener() quedaron anidadas dentro de la función principal (solo reindentación).

index.html y styles.css

index.html

- Canvas de 400×400 a 1000×400.
- Nueva barra de Salud (HP).
- Fragmentos de “0 / 5” a “0 / 10”; estado inicial “Dormida”; acentos corregidos.
- Botones fuera del panel de estado.

styles.css

- Panel de estado de 240px, align-items: stretch y resplandor en el canvas.

Explicación Obstáculos:



El proyecto simula un agente inteligente reflejo basado en modelo (C.C.) que explora un mapa nevado recolectando fragmentos de espejo, mientras interactúa con distintos obstáculos: uno que la ayuda, uno que la daña, uno que la sorprende, uno que bloquea su paso, y terreno que puede matarla si tiene mala suerte.

Obstáculo	Tipo	Efecto
Dragon	Móvil, dañino	Hace daño si C.C. se acerca; rota entre 4 direcciones

Claire (aliada)	Estático, benéfico	Restaura energía y salud al pisar su casilla
Castillo	Estático, bloqueante	Estructura grande que C.C. no puede atravesar
Brujo	Estático, dañino	Quita energia y salud al pisar su casilla
Árboles	Estático, bloqueante	Forman muros/corredores que C.C. debe rodear
Trampa de hielo	Oculto, dañino	Invisible hasta pisarla; daña una sola vez
Río de hielo	Terreno	Cuesta más energía cruzarlo; puede romperse y matar a C.C.

El Dragón (obstáculo móvil)

El dragón usa una imagen real (sprite sheet de 3 columnas x 4 filas) en vez de la técnica de letras que usa C.C. Ocupa 2x2 casillas y rota entre 4 direcciones cada 4 ciclos. Si C.C. se acerca a distancia ≤1, recibe daño continuo.






```
actualizar(agente) {
    const casillasDrag = this.obtenerCasillasOcupadas();
    const estaCerca = casillasDrag.some(c => {
        const dist = Math.abs(agente.x - c.x) + Math.abs(agente.y - c.y);
        return dist <= 1;
    });

    if (estaCerca) {
        agente.recibirDano(this.danoPorTurno, this.x, this.y);
    }

    this.animar();
}
```

Claire, la aliada (obstáculo estático benéfico)

Claire es un personaje que usa una imagen real (sprite de guerrera recortado de un sprite sheet de 4 direcciones), pero como no se mueve de lugar, solo usa el frame de la dirección "ABAJO". Tiene un balanceo visual sutil con `ctx.rotate()` para no verse congelada.

```
// aliada.js — fragmento clave
actualizar(agente) {
    const estaEncima = (agente.x === this.x && agente.y === this.y);
    if (estaEncima) {
                        agente.energia = Math.min(100, agente.energia +
this.restauraEnergiaPorTurno);
        agente.salud = Math.min(100, agente.salud + this.restauraSaludPorTurno);
    }
```

```
}
```

El Castillo (obstáculo bloqueante grande)

El castillo es una imagen grande (dibujada sobre varias casillas), y a diferencia de los obstáculos puramente decorativos, sí bloquea el paso real: define qué casillas ocupa su "cuerpo", dejando libre únicamente su entrada.

```
// castillo.js — fragmento clave
obtenerCasillasOcupadas() {
    return [
        { x: this.x - 1, y: this.y - 1 },
        { x: this.x,     y: this.y - 1 },
        { x: this.x + 1, y: this.y - 1 },
        { x: this.x - 1, y: this.y },
        { x: this.x + 1, y: this.y }
        // la casilla (this.x, this.y) NO está bloqueada:
        // es la entrada, siempre transitable
    ];
}
```

El Brujo (obstáculo estático dañino)



Misma estructura que Claire pero en negativo: resta energía y salud en vez de sumar. Usa `agente.recibirDano()` en vez de restar salud directamente, para reutilizar la lógica de muerte/respawn que ya existía.

```
actualizar(agente) {
    if (agente.estaMuerta) return;

    const estaEncima = (agente.x === this.x && agente.y === this.y);
    if (estaEncima) {
        agente.energia = Math.max(0, agente.energia - this.quitaEnergiaPorTurno);
        agente.recibirDano(this.quitaSaludPorTurno, this.x, this.y);
        agente.estado = `🔮 El brujo esta drenando a C.C. (Energía: ${agente.energia}%)`;
    }
}
```

La Trampa de hielo (obstáculo oculto)

A diferencia de los demás, este obstáculo no se dibuja hasta que C.C. lo pisa por primera vez — es literalmente invisible para los 5 sentidos de C.C. Al activarse: hace daño una sola vez, se revela visualmente, y se registra en la memoria de C.C. (`mapaMental`) como zona peligrosa, para que la evite en el futuro.




```
actualizar(agente) {
    if (agente.estaMuerta) return;
    const estaEncima = (agente.x === this.x && agente.y === this.y);
    if (!estaEncima) return;

    if (!this.revelada) {
        this.revelada = true;
        agente.estado = `❄️✨ ¡C.C. activó una trampa de hielo oculta!`;

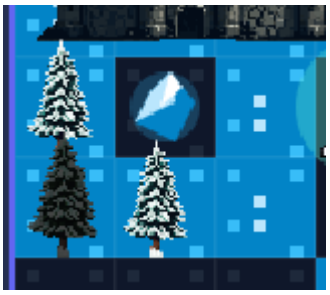
        const clave = `${this.x},${this.y}`;
        if (!agente.mapaMental[clave]) {
            agente.mapaMental[clave] = { revelado: true, visitas: 0, costo: 0.5, tieneCristal: false, esZonaFuego: false };
        }
        agente.mapaMental[clave].esZonaFuego = true;

        agente.recibirDano(this.danoSalud, this.x, this.y);
        this._flashTicks = 3;
    }
}

dibujar(ctx) {
    if (!this.revelada) return;
```

Los Árboles (muros y corredores en "L")

Los árboles reutilizan el mismo sistema de bloqueo que el castillo (`obstaculosBloqueantes` en `agente.js`), pero agrupados estratégicamente: algunos forman "muros" que tapan corredores completos, y otros forman figuras en "L" alrededor de fragmentos específicos, dejando siempre una sola entrada libre, para obligar a C.C. a encontrar la ruta correcta en vez de solo esquivar un obstáculo suelto.



```
// agente.js — el sistema que hace que los árboles (y el castillo)
// realmente bloqueen el paso, no solo se vean ahí
registrarObstaculosBloqueantes(lista) {
    this.obstaculosBloqueantes = lista;
}
```

```
estaBloqueado(x, y) {
    return this.obstaculosBloqueantes.some(o => o.x === x && o.y === y);
}
```

```
// Dentro de seleccionarMejorPaso(): cualquier dirección bloqueada
// se descarta antes de siquiera calcular su puntaje
if (nx >= 0 && nx < mapa.columnas && ny >= 0 && ny < mapa.filas && !this.estaBloqueado(nx, ny)) {
    // ...solo aquí se evalúa el movimiento...
}
```

El río de hielo y el hielo que se rompe



El "río" no es un obstáculo nuevo — es terreno de hielo normal (ya existía en el juego) puesto estratégicamente para separar zonas del mapa. El hielo no bloquea el paso, pero cuesta más energía cruzarlo, así que el algoritmo de C.C. prefiere evitarlo cuando hay una ruta más barata disponible.

Le agregamos un riesgo adicional: cada vez que C.C. pisa hielo, hay 10% de probabilidad de que se rompa bajo sus pies.

```
if (tactoActual.casillaActual === TIPO_CASILLA.HIELO) {
    if (Math.random() < 0.10) this.ahogarseEnHielo();
}
```

```
ahogarseEnHielo() {
    if (this.estaMuerta) return;
    this.estaMuerta = true;
    this.salud = 0;
    this.cayendoHielo = 8;
    this.estado = "❄💧 ¡El hielo se rompió bajo los pies de C.C.! Cayó al agua helada... Reapareciendo en 10 segundos...";
    setTimeout(() => { this.reaparecer(); this.estaMuerta = false; }, 10000);
}
```



El efecto visual (ondas de agua, grietas, y la calavera flotando) se dibuja completamente con geometría de Canvas (círculos, líneas, triángulos) — sin usar ninguna imagen nueva.

cambios realizados en los agentes

Problemas

Se presentaron algunos problemas en cuanto al movimiento de los agentes, aliada y brujo, por lo que implementamos la toma de decisiones con la evaluación de la función heurística.

se implementó una lógica a la aliada mediante distanciamanhattan, para que la aliada sirviera como "guardiana" del agente principal, se implementó un método donde siempre que aliada esté cerca del agente principal el brujo huye, y cuando aliada esté lejos, el brujo ataca al CC

También agregamos los sentidos de vista y olfato para estos dos agentes, la vista les ayuda a observar su alrededor y poder saber lo que exploran y guardándolo mientras caminan, el olfato les sirve para saber hasta dónde está el uno del otro y así el brujo pueda escapar.

Aliada:

```
reglaReflejo(brujo, dragon, agente) {
    if (this.estaMuerta) return 'MUERTA';

    // Captura sensorial de distancias
    const distBrujo = this.percibirDistanciaManhattan(brujo);
    const distDragon = this.percibirDistanciaManhattan(dragon);

    // 1. TACTO / ALCANCE DE ATAQUE: Distancia <= 1 (Casilla adyacente)
    if (distBrujo <= 1) return 'ATACAR_BRUJO';
    if (distDragon <= 1) return 'ATACAR_DRAGON';

    // 2. VISTA / OLFATO: Rango de detección de hasta 6 casillas
    if (distBrujo <= 6) return 'PERSEGUIR_BRUJO';
    if (distDragon <= 6) return 'PERSEGUIR_DRAGON';

    // 3. REGLA POR DEFECTO: Si no percibe enemigos, acompaña al Agente C.C.
    return 'ACOMPANAR_AGENTE';
}
```

Brujo:

```
reglaReflejo(aliada, agente) {
    if (this.estaMuerto) return 'MUERTO';

    // VISTA / OLFATO: Si la Aliada entra en un radio de 4 casillas, la percibe y de
    const distAliada = this.percibirDistanciaManhattan(aliada);
    if (distAliada <= 4) {
        return 'HUIR_DE_ALIADA';
    }

    // VISTA / OLFATO: Si el Agente Principal está a 6 casillas o menos, decide asec
    const distAgente = this.percibirDistanciaManhattan(agente);
    if (distAgente <= 6) {
        return 'ASECHAR_AGENTE';
    }

    return 'EXPLORAR_INTELIGENTE';
}
```

la aliada tiene un método aún borrador donde golpea al brujo cuando este cerca de él, pero aún falta perfeccionarlo, también añadiremos que golpee al dragón, para que C.C pueda escapar.

```
// Dentro de actuar(accion, mapa, agente, dragon, brujo) en aliada.js

case 'ATACAR_BRUJO':
    this.estado = "¡Atacando al Brujo!";

    // Invocación del golpe al Brujo pasándole la cantidad de daño
    if (brujo && typeof brujo.recibirDano === 'function') {
        brujo.recibirDano(this.danoAtaque, this.x, this.y);
    }
    break;
```

agregamos un método evaluarmovimiento donde calcula el puntaje de costo de una casilla vecina, donde el menor puntaje es mejor la opción.

El brujo también tiene un método donde recibe daño de la aliada, que aun falta por perfeccionar también.

```
// Método en brujo.js

recibirDano(puntos, origenX, origenY) {
    if (this.estaMuerto) return;

    // 1. Descuenta los puntos de salud sin bajar de 0
    this.salud = Math.max(0, this.salud - puntos);

    // 2. Activa el destello rojo por 3 fotogramas en el método dibujar()
    this.turnosEfectoImpacto = 3;

    // 3. Comprueba estado de salud y actualización
    if (this.salud <= 0) {
        this.estaMuerto = true;
        this.salud = 0;
        this.estado = "El Brujo ha sido derrotado";
    } else {
        this.estado = `¡Brujo atacado! HP restante: \({this.salud}/\){this.saludMax}
    }
}
```

Al agregar el algoritmo estrella, los agentes razonan y toman mejores decisiones para su movimiento. Detectamos que por las sumas y restas simples, llega un momento donde los agentes empiezan a hacer los mismos movimientos pero con el algoritmo estrella se controla eso.

Conclusiones

Es interesante el poder ir desarrollando un agente que vaya evolucionando, en un inicio solamente se mueva solo sin rumbo con pura suerte a que sea capaz de recordar cada punto por el que ha pasado, recordando caminos por los que ya tránsito y no volver a pasar, aplicarle un algoritmo estrella para que calcule el costo de cada movimiento que hace y mucho más. Por el momento solo está en un estado t-1 pero cuando llegué a t+1 será mucho más interesante poder observar cada decisión que toma y el por qué.

Referencias bibliográficas

[1] X. Qiao, «PathFinding.js: A visual representation of pathfinding algorithms in JavaScript,» GitHub Pages, 2026. [En línea]. Disponible en: https://qiao.github.io/PathFinding.js/visual/.

[2] A. Patel, «Introduction to A Pathfinding,» Red Blob Games, [En línea]. Disponible en: https://qiao.github.io/PathFinding.js/visual/.*

[3] A. Patel, «Introduction to the A Algorithm,» Red Blob Games, 2014. [En línea]. Disponible en: https://www.redblobgames.com/pathfinding/a-star/introduction.html.*